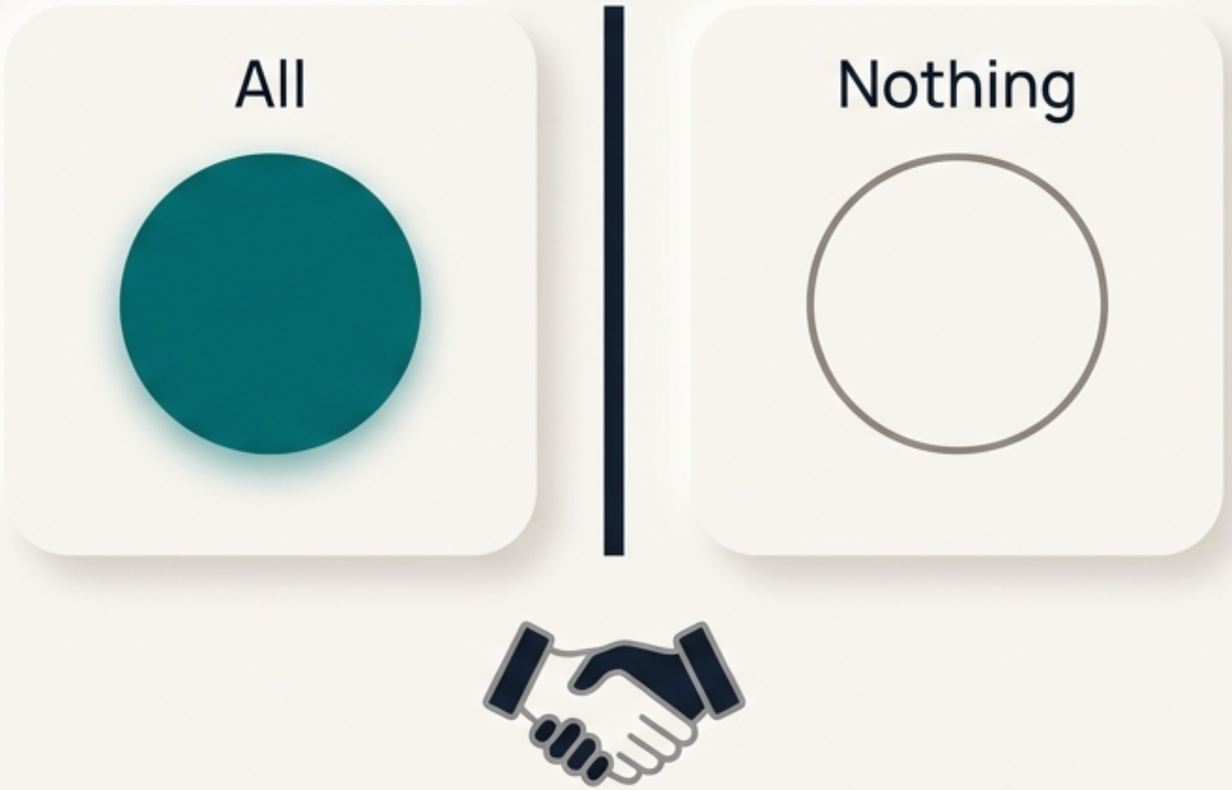
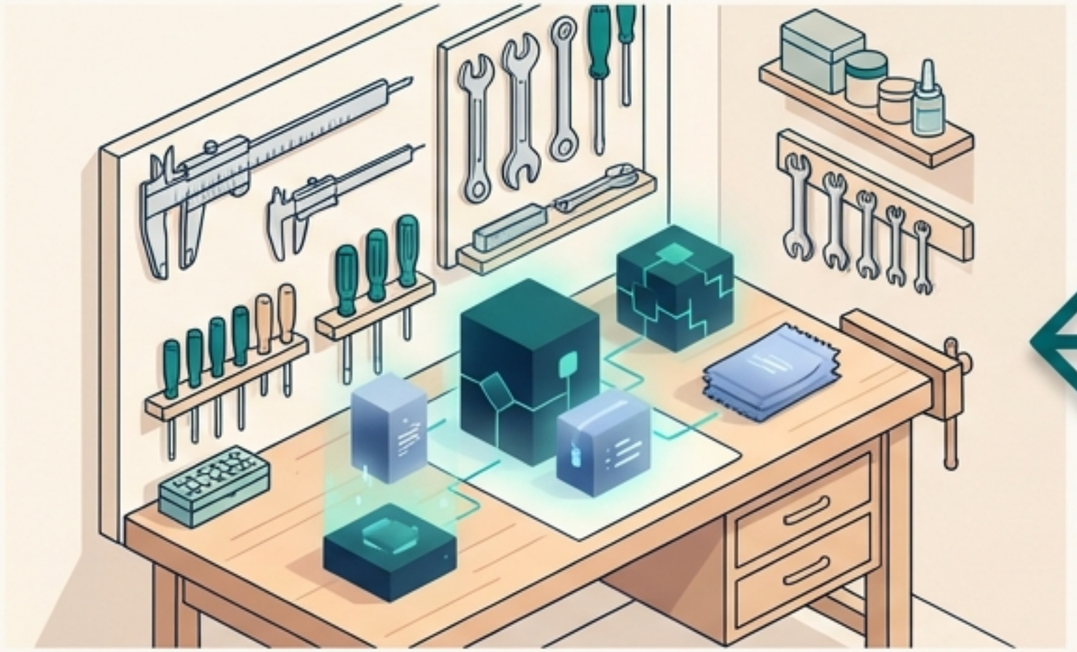
Every Database Transaction Is a Promise

Think of it as an “All-or-Nothing” contract. When you ask a database to do something, it must complete the entire job, or do none of it at all. There is no middle ground. This property is called **atomicity**.



The Two Homes for Your Data



The Cache (The Workbench)

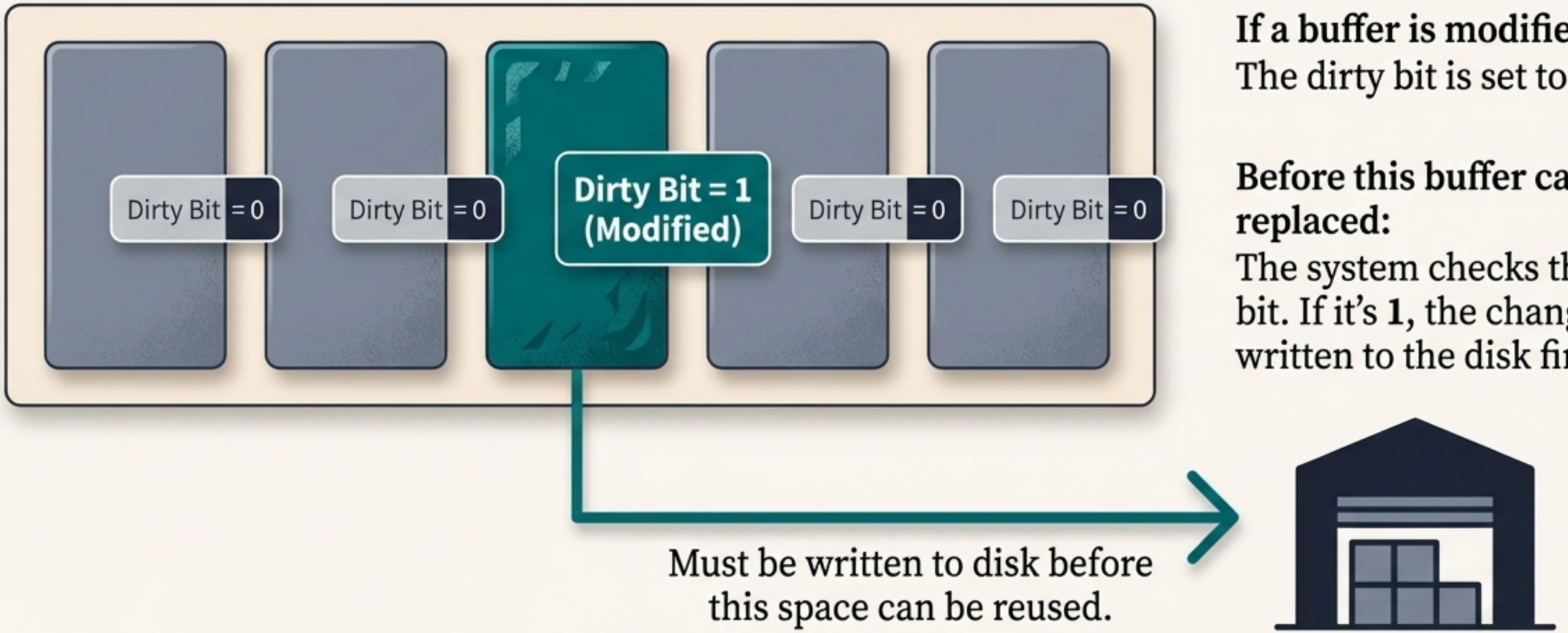
A collection of in-memory buffers. It's super fast, like a workbench right next to you. This is where active work happens.

The Disk (The Warehouse)

The permanent storage. It's slower to access, like walking to a giant warehouse to get a tool. This is where data is kept safe for the long term.

Keeping Track of Changes on the Workbench

The database needs a simple way to know if a piece of data in the cache has been changed but not yet saved to the disk. It uses a flag called a **dirty bit.**



If a buffer is modified:
The dirty bit is set to **1.**

Before this buffer can be replaced:
The system checks the dirty bit. If it's **1**, the changes are written to the disk first.

Then, the Unexpected Happens: A System Crash

Power fails.
A bug occurs.
The system crashes.

What happens to:

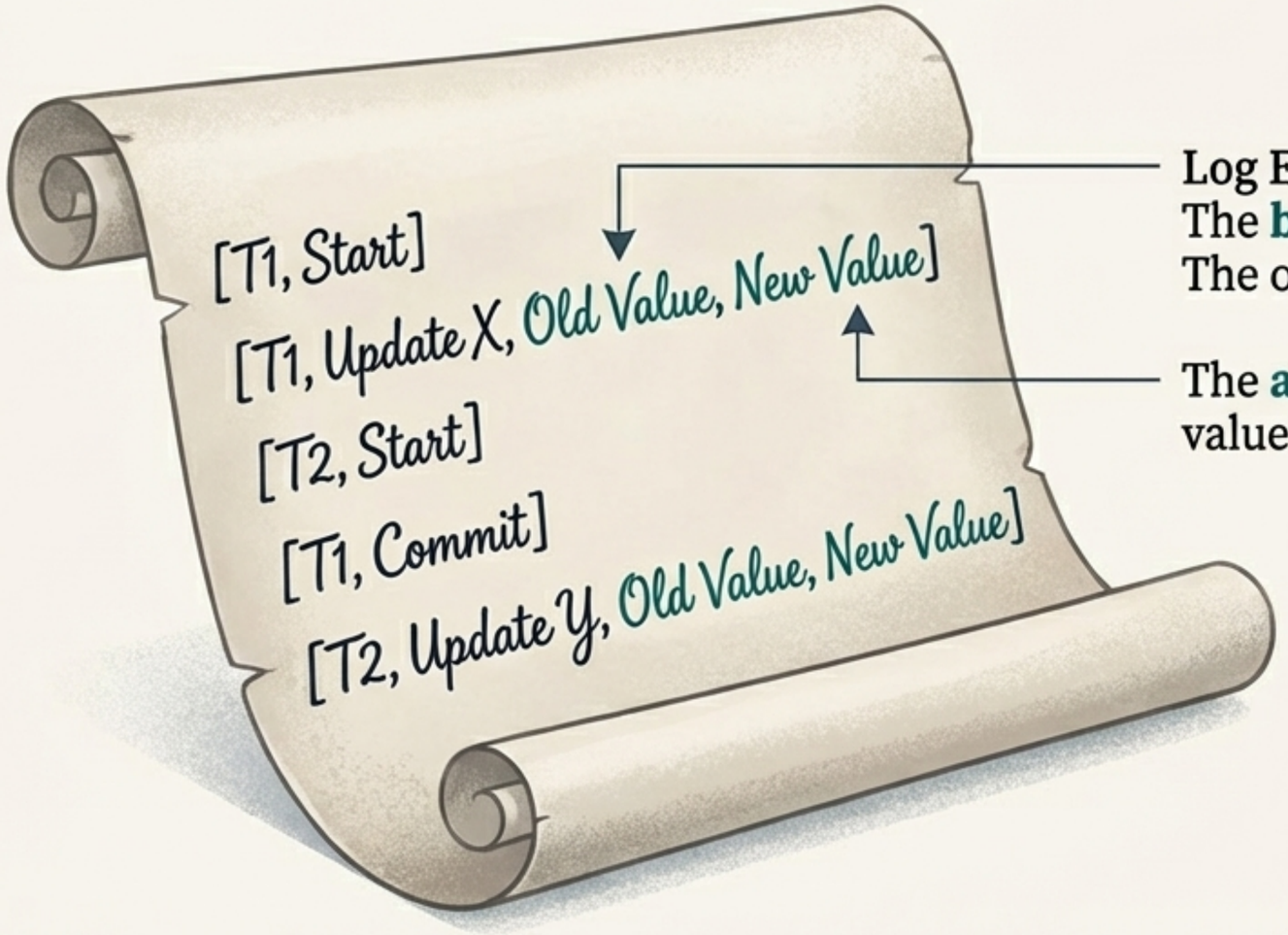
- Transactions that were finished but whose changes were still only in the fast (but temporary) cache?

- Transactions that were only halfway done when the system went down?

Is our data lost forever?

The Safety Journal: A Perfect Memory

No. Because the database was keeping a secret diary. The **System Log** meticulously records every single action before it happens. It’s the flight recorder for your data.



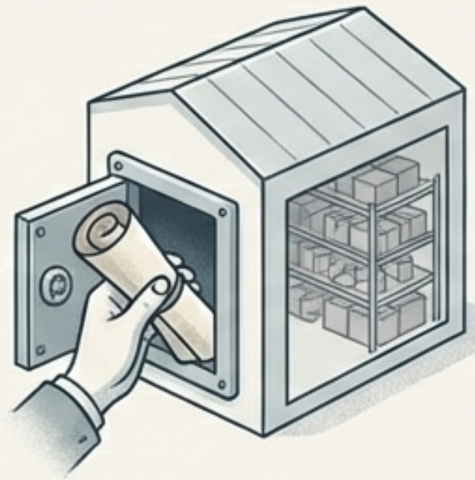
Log Entries Include:
The **before-image** (BFIM): The old value of a data item.

The **after-image**: The new value of a data item.

The Golden Rule: Always Write to the Journal First

The entire recovery system is built on one non-negotiable principle: the **Write-Ahead Logging (WAL)** protocol.

MUST DO FIRST



2. Force Log to Disk

3. Write to Database Disk

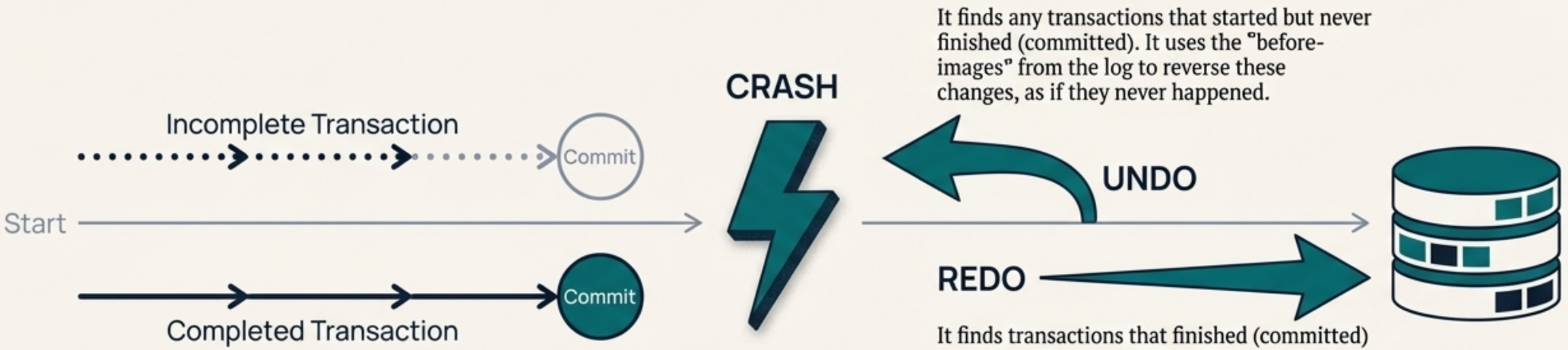
The Rule, Stated Simply:

Before a change is ever made to the database itself (on disk), the log record describing that change *must* be written to the disk first.

- **For an UNDO:** The 'before-image' must be safely in the log before the data is overwritten on disk.
- **For a REDO:** The 'after-image' must be in the log before a transaction can be considered complete ('committed').

Waking Up: How the Database Heals Itself

When the system restarts, it reads its Safety Journal (the log) to figure out what happened and fix it.



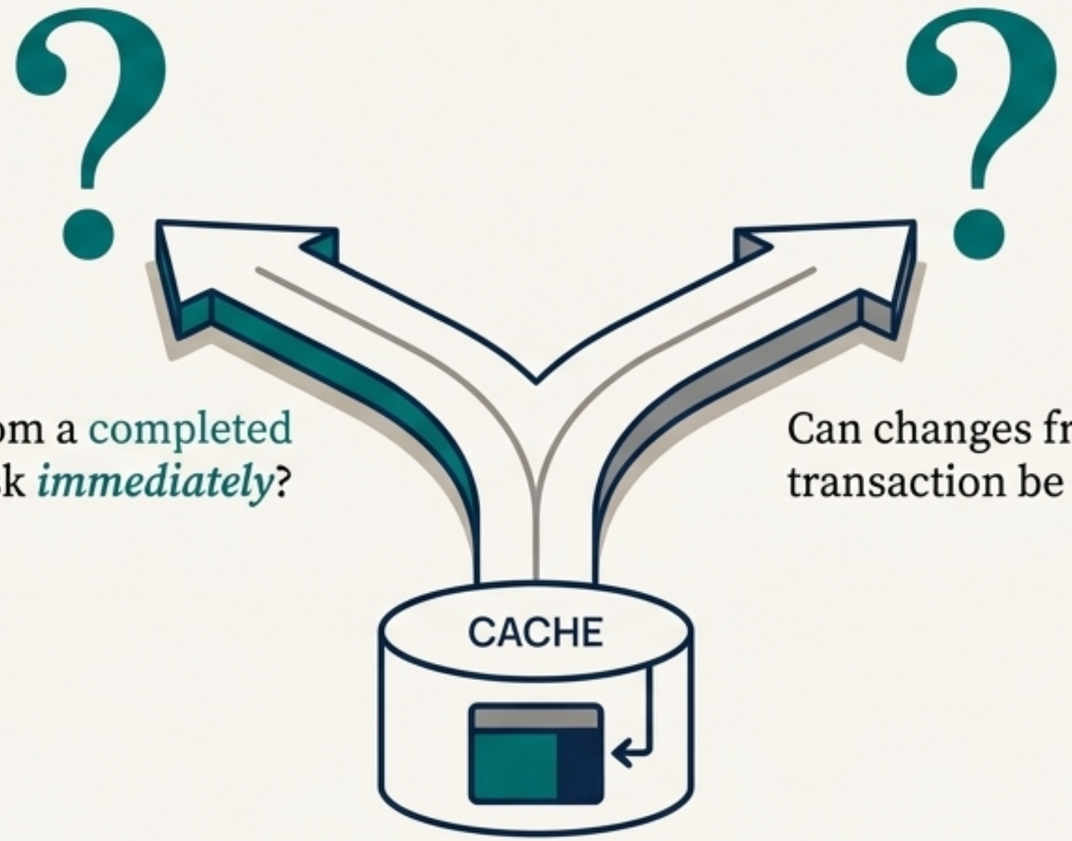
Idempotent

Both UNDO and REDO operations are idempotent. Running them multiple times has the same result as running them once, which guarantees the recovery process is safe even if it's interrupted.

The Rules of the Workspace: How Data Moves from Cache to Disk

To balance safety and speed, databases use a sophisticated set of rules to decide *when* to move changed data from the fast cache to the permanent disk.

Must all changes from a completed transaction be written to disk *immediately*?



Can changes from an *unfinished* transaction be written to disk?

Policy Decision 1: Force vs. No-Force

This policy decides if a transaction's committed changes must go to the disk right away.

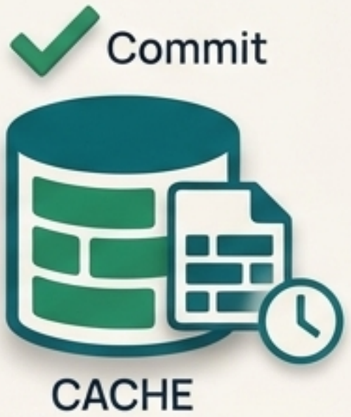
FORCE Approach

What it is: All pages updated by a transaction are immediately written to disk when it commits.

Pro: Simple. A committed transaction is definitely safe on disk.

Con: Slow. Lots of disk I/O, even if the same data is needed again right away.

NO-FORCE Approach



What it is: The system is *not* required to write all updated pages to disk immediately upon commit.

Pro: Fast and efficient. Reduces disk I/O.

Con: More complex. Requires REDO logic during recovery to re-apply any committed changes that were still in the cache during a crash.

Policy Decision 2: Steal vs. No-Steal

This policy decides if the database is allowed to write uncommitted changes to disk.

NO-STEAL Approach

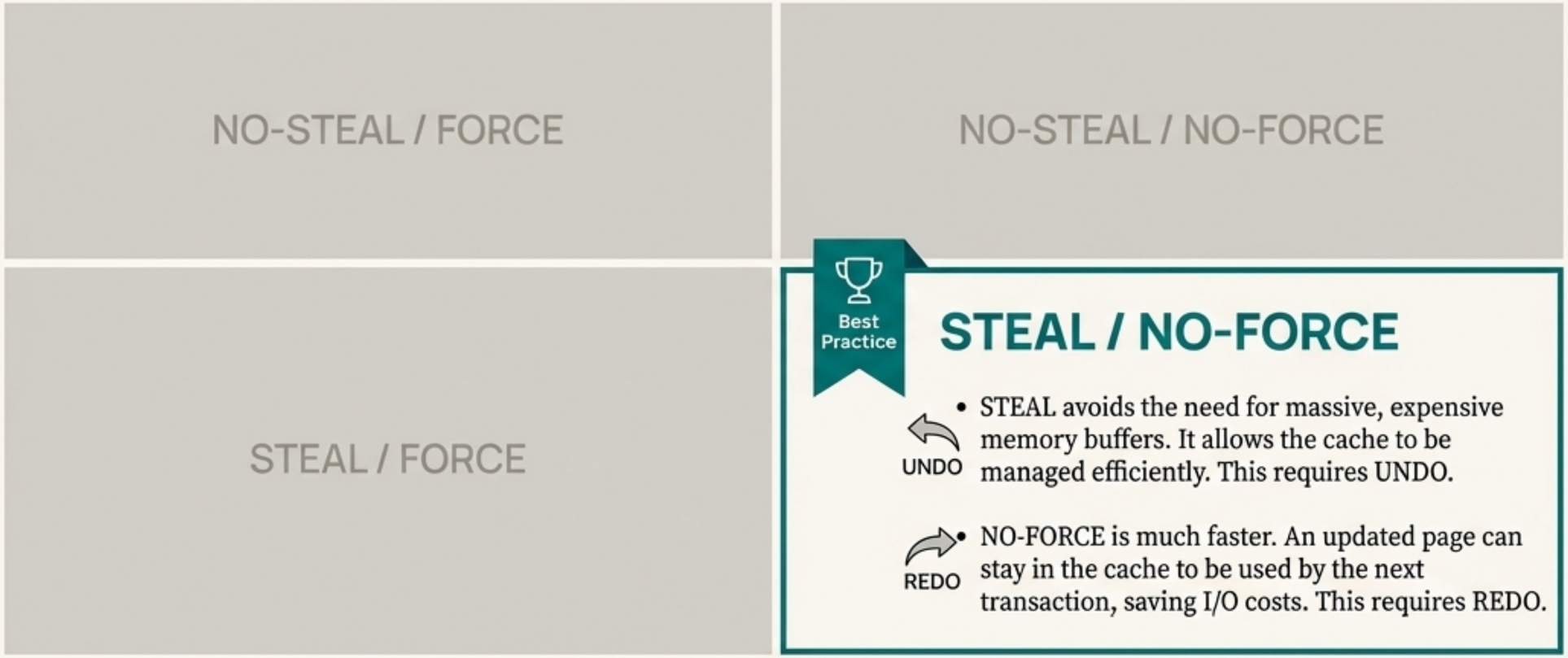
- **What it is:** A cache buffer with changes from an uncommitted transaction *cannot* be written to disk.
- **Pro:** Simple. No need for UNDO logic for data on disk, as unfinished work is never there.
- **Con:** Requires a very large cache to hold all changes from long-running transactions. Impractical.

STEAL Approach

- **What it is:** The system is *allowed* to write updated buffers to disk even before the transaction commits.
- **Pro:** Very efficient with memory. Doesn't need huge buffers.
- **Con:** More complex. Requires UNDO logic during recovery to roll back any uncommitted changes that made it to disk before a crash.

The Winning Strategy: Steal and No-Force

Almost all modern database systems use a STEAL/NO-FORCE strategy. It provides the best balance of performance and practicality.



The Bottom Line: This combination is high-performance, but it relies on a robust recovery system with both UNDO and REDO capabilities, powered by the Write-Ahead Log.

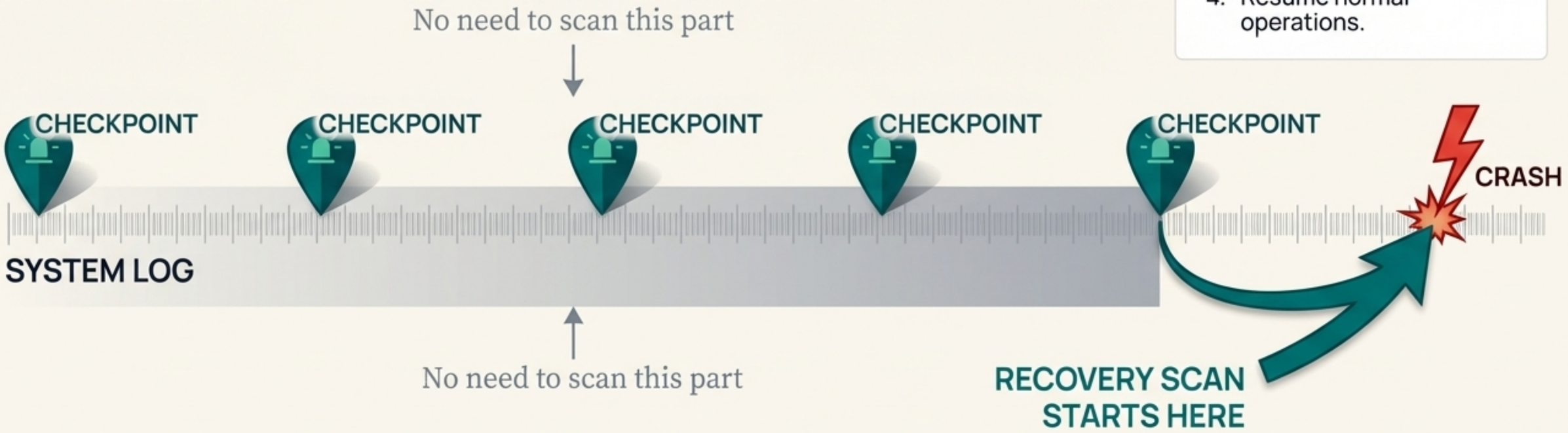
Making Recovery Faster with Checkpoints

If a database runs for weeks, its log can become enormous. Re-reading the entire log after a crash would take forever.

The solution is a **Checkpoint**, like creating a "save point" in a video game.

The Process

1. Temporarily suspend new transactions.
2. Force-write all modified ("dirty") buffers from the cache to the disk.
3. Write a `[checkpoint]` record into the log and force it to disk.
4. Resume normal operations.



A Problem to Avoid: The Domino Effect

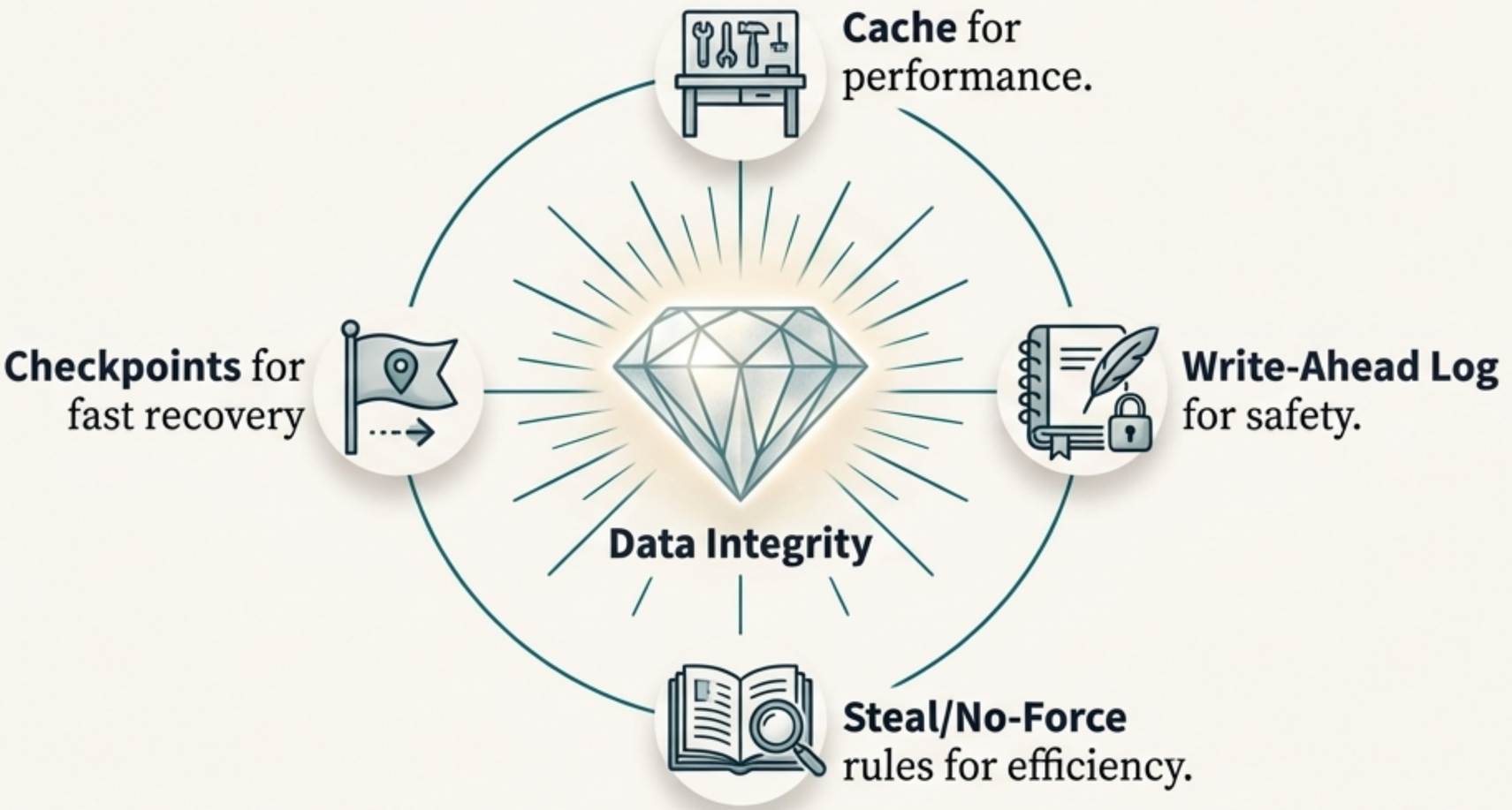
What if Transaction B reads data that was written by Transaction A, and then Transaction A fails and must be rolled back? This would trigger a Cascading Rollback. Transaction B would have to be rolled back, and any other transaction that read its data would also have to be rolled back, and so on.

Good News

This is a recognized problem. Almost all modern recovery protocols are designed to create “recoverable schedules” that completely prevent cascading rollbacks from ever happening.

The Guarantee: Your Data is Always Safe

From a simple promise of “all-or-nothing,” a sophisticated system works constantly behind the scenes.



Together, these techniques ensure that no matter what happens—even a sudden crash—the database can wake up, heal itself, and guarantee that your data is in a consistent and correct state. The promise is always kept.